
accu
On Sea
O(1) or O(no-no-no)
Mastering the unordered_map
Kevin B. Carpenter

For decades, we taught C++ with two containers:

`std::vector` and `std::map`

That's wrong.

If `std::vector` is our default sequence container...

What is our default *associative* container?

`std::unordered_map`

The #2 container in modern C++

1. **WHY** it's dominant (the data)
2. **HOW** it works (the internals)
3. **WHEN** it fails (the pitfalls)

Agenda

The Showdown	`map` vs `unordered_map`
Under the Hood	How a hash table really works
Three Pitfalls	Hashing, Collisions, Rehashing
HashDoS	When collisions become a CVE
Beyond	`flat_map`, heterogeneous lookup, open addressing
Memory & Profiling	Cache, allocators, perf, concurrency
Decision Tree	Your new framework

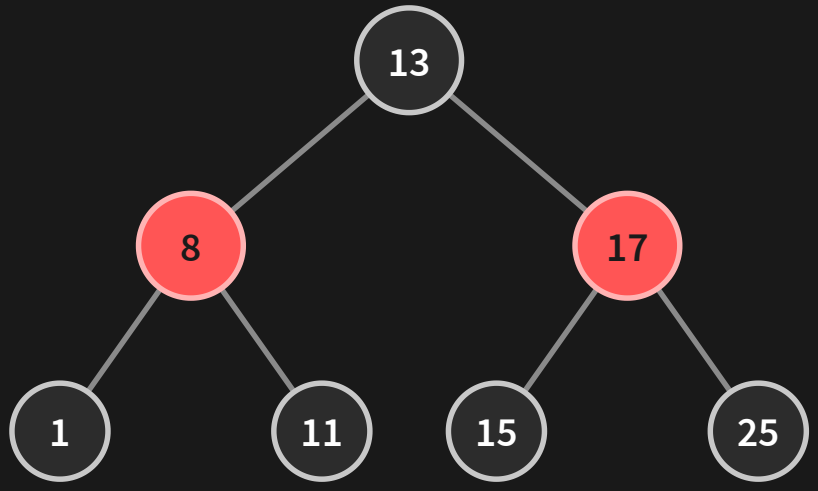
Round 1

`map` vs. `unordered_map`

`std::map` — Red-Black Tree

- Guaranteed `O(log n)` lookup, insert, delete
- Keys always sorted
- Node-based: pointer-chasing, cache-unfriendly
- Stable iterators/references across all operations

What is a Red-Black Tree?



● red ◯ black nil sentinels not shown

Self-balancing → height stays ≈ `log n`

⇒ guaranteed `O(log n)` lookup / insert / erase

Every node is a separate **heap allocation**

Coloring keeps the longest path ≤ 2× the shortest — that's the balance.

`std::unordered_map` — Hash Table

- Average `O(1)` lookup, insert, delete
- Keys unordered
- Bucket-based: separate chaining (linked lists)
- *"Average"* is doing heavy lifting in that sentence

The Benchmark

```
constexpr int N = 1'000'000;

using Map_t = std::map<std::string, uint32_t>;
using Hash_t = std::unordered_map<std::string, uint32_t>;
Map_t ordered;
Hash_t unordered;

for (int i = 0; i < N; ++i) {
    std::string txn_id = "TXN" + std::to_string(i);
    ordered[txn_id] = i * 100;   // amount in cents
    unordered[txn_id] = i * 100;
}

// Benchmark: look up all N keys
auto start = high_resolution_clock::now();
for (int i = 0; i < N; ++i) {
    std::string key = "TXN" + std::to_string(i);
    volatile auto it = ordered.find(key);
}
auto map_time = high_resolution_clock::now() - start;
```

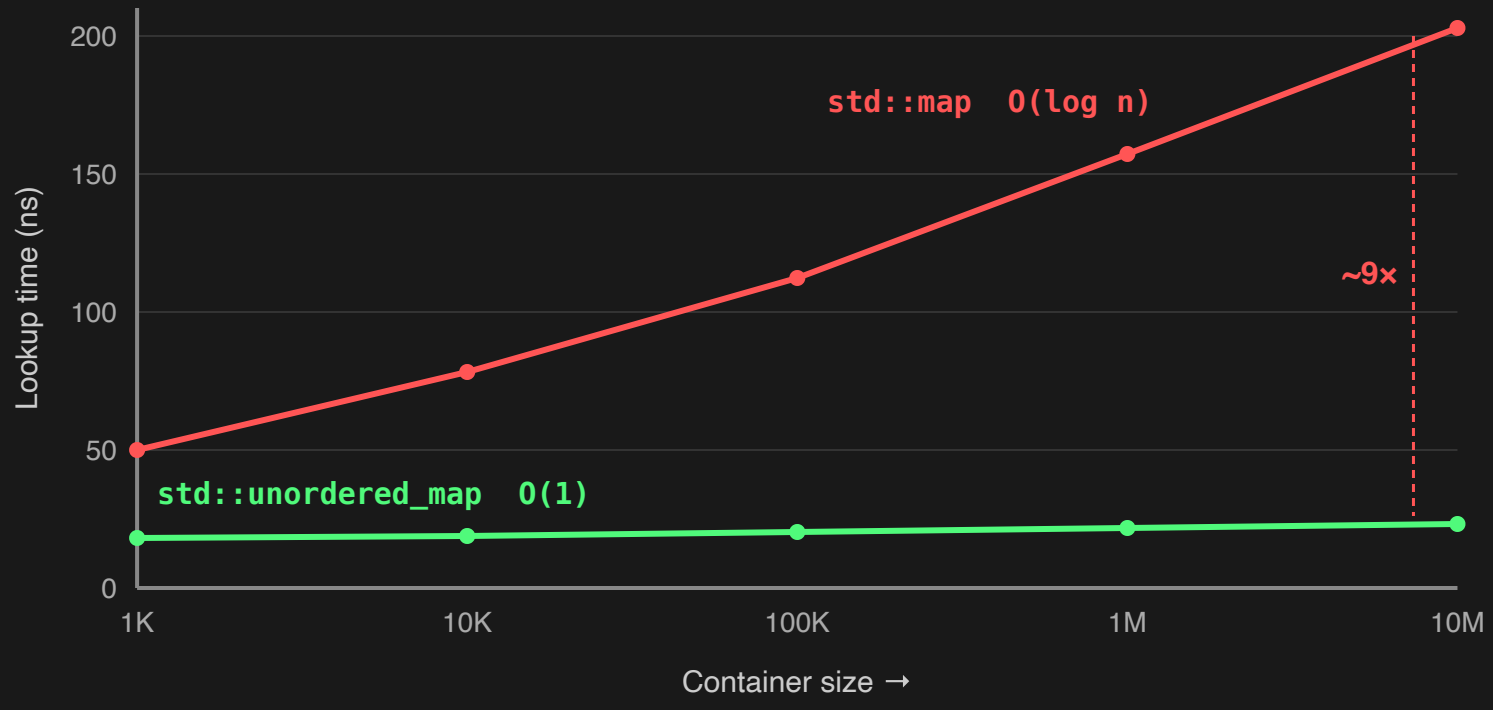
src/02_benchmark_simple_txn.cpp

Auth Lookup: map vs unordered_map

Auth Count	std::map (ns)	unordered_map (ns)	Speedup
1,000	~50	~18	2.9x
10,000	~78	~19	4.1x
100,000	~112	~20	5.5x
1,000,000	~157	~22	7.0x
10,000,000	~203	~23	8.8x

src/01_benchmark_map_vs_unordered_txn.cpp

Lookup Time vs Container Size



measured lookup latency · src/01_benchmark_map_vs_unordered_txn.cpp

At 10 million items, `map` lookup is

~9x slower

At 10,000 TPS, nanoseconds become milliseconds become SLA breaches.

One honest caveat

`O(1)` is in the **number of elements** — not free.

`unordered_map`: every lookup **hashes + compares the whole key** → `O(k)` in key length *k*

`map`: `O(k · log n)` — the comparisons add up too

For long string keys, hashing cost is real. The win is in *n*, not in *k*.

Under the Hood

How `std::unordered_map` really works

The Pipeline

Step 1: Key → `std::hash` → `size_t`

Step 2: Hash → `hash % bucket_count()` → Bucket Index

Step 3: Bucket → Linear scan → Data

Step 1: Key → Hash

```cpp
size_t hash = std::hash<std::string>{}(mid);
size_t bucket = hash % merchant_cache.bucket_count();
```

"MID_100001" → hash: 149238475610 23 → bucket: 3

"MID_200045" → hash: 827364519283 74 → bucket: 7

src/03_under_the_hood_txn.cpp

Step 2: The Bucket Array

```
std::cout << "Size: " << map.size();
std::cout << "Bucket count: " << map.bucket_count();
std::cout << "Load factor: " << map.load_factor();
std::cout << "Max load factor: " << map.max_load_factor();
```

```
Bucket 0: []
Bucket 1: ["MID_300078"]
Bucket 2: []
Bucket 3: ["MID_100001"] → ["MID_500099"]
Bucket 4: []
Bucket 5: ["MID_200045"]
```

src/03_under_the_hood_txn.cpp

Separate Chaining

Bucket 3 (head of linked list):

```
"MID_100001" →
   "MID_500099" →
   null
```

Each bucket is the head of a **linked list**.

When two keys hash to the same bucket, they get **chained**.

This isn't an accident: the standard's bucket interface + reference stability across rehash effectively *mandate* node-based chaining. (Hold that thought — it's why faster maps exist.)

`O(1)` assumes each bucket has ~1 item.

`O(n)` happens when one bucket has *ALL* items.

Load Factor

```
load_factor = size() / bucket_count()
```

When `load_factor > max_load_factor`
(default: **1.0**)

> **REHASH:** Allocate new bucket array → re-hash every element → re-insert all

Pitfall 0: It's in the name

The next three pitfalls cost you *performance*.

This one costs you **correctness** — and it bites first.

A real auth service

A per-request `std::map` is iterated **in order** to build the wire message…

```
std::string requestMapToMML( std::map<std::string,std::string>& req ) {
    std::string mml;
    for ( const auto& field : req ) {        // sorted iteration
        mml.append( toMmlTag(field.first) );
        mml.append( field.second );
        mml += MML_FIELD_SEP;
    }
    mml += Crypto::SHA1( mml );              // order is now in the hash
    return mml;
}
```

Swap to `unordered_map` → fields reorder → bytes change → SHA1 changes → the switch rejects the message.

distilled from a real payment-auth service · src/22_ordering_is_load_bearing.cpp

Same data, one swap

```
std::map           AMOUNT5000|AUTH_GUIDA1B2|BATCH_ID42|CARD_BIN411111|...
                   hash = 6012151533857893158   ACCEPTED

std::unordered_map BATCH_ID42|TRACEZZ9|TERM_IDTERM07|CURRENCYUSD|...
                   hash = 64119505218196699 82   REJECTED — wire protocol broken
```

No bad hash. No collision. No rehash. A pure **correctness** bug.

Hides anywhere iteration order is load-bearing: wire formats, signatures/HMACs, golden-file tests, dedup keys.

When O(1) Becomes O(n)

…and now the three *performance* pitfalls

Pitfall 1: The Bad Hash

Pitfall 2: The Collision Catastrophe

Pitfall 3: The Hidden Rehash

Pitfall 1: The Bad Hash

You need to hash a custom struct.

What could go wrong?

The Transaction Struct

```cpp
struct Transaction {
    std::string merchant_id;
    std::string card_bin;
    uint32_t amount_cents;
    std::string currency;

    bool operator==(const Transaction& other) const {
        return merchant_id == other.merchant_id
            && card_bin == other.card_bin
            && amount_cents == other.amount_cents
            && currency == other.currency;
    }
};
```

src/04_pitfall1_bad_hash_txn.cpp

Bad Hash vs Good Hash

BAD

```
struct BadHash {
  size_t operator()(
    const Transaction& t) const {
    return std::hash<std::string>{}
      (t.currency);
  }
};
// Only hashes currency
// 99% "USD" → collision!
```

GOOD

```
struct GoodHash {
  size_t operator()(
    const Transaction& t) const {
    size_t seed = 0;
    // Order-dependent mixing:
    mix(seed, t.merchant_id);
    mix(seed, t.card_bin);
    mix(seed, t.amount_cents);
    mix(seed, t.currency);
    return seed;
  }
};
```

Why not plain `seed ^= hash(field)`? XOR is commutative — swap merchant/BIN and you get the *same* hash. `mix()` is on the next slide.

src/04_pitfall1_bad_hash_txn.cpp

The Pro-Tip: hash_combine

```cpp
template <typename T>
void hash_combine(size_t& seed, const T& val) {
    const size_t GOLDEN = 0x9e3779b9;
    seed ^= std::hash<T>{}(val) + GOLDEN
            + (seed << 6) + (seed >> 2);
}

template <typename... Args>
inline size_t hash_values(const Args&... args) {
    size_t seed = 0;
    (hash_combine(seed, args), ...);  // C++17 fold expression
    return seed;
}
```

```cpp
struct TransactionHash {
    size_t operator()(const Transaction& t) const {
        return hash_values(
            t.merchant_id, t.card_bin, t.amount_cents,
            t.currency, t.timestamp_ms);
    }
};
```

src/05_pitfall1_hash_combine_txn.cpp

Pitfall 2: The Collision Catastrophe

What happens when **every** key hashes to the same bucket?

The Terrible Hash

```
struct TerribleAuthHash {
    size_t operator()(const std::string&) const {
        return 42;  // Every auth code goes to bucket 42
    }
};
```

```
using IntMap_t = std::map<std::string, uint32_t>;
using GoodHash_t = std::unordered_map<
    std::string, uint32_t>;
using BadHash_t = std::unordered_map<
    std::string, uint32_t, TerribleAuthHash>;
IntMap_t ordered;
GoodHash_t good_hash;
BadHash_t bad_hash;
```

src/06_pitfall2_collision_catastrophe_txn.cpp

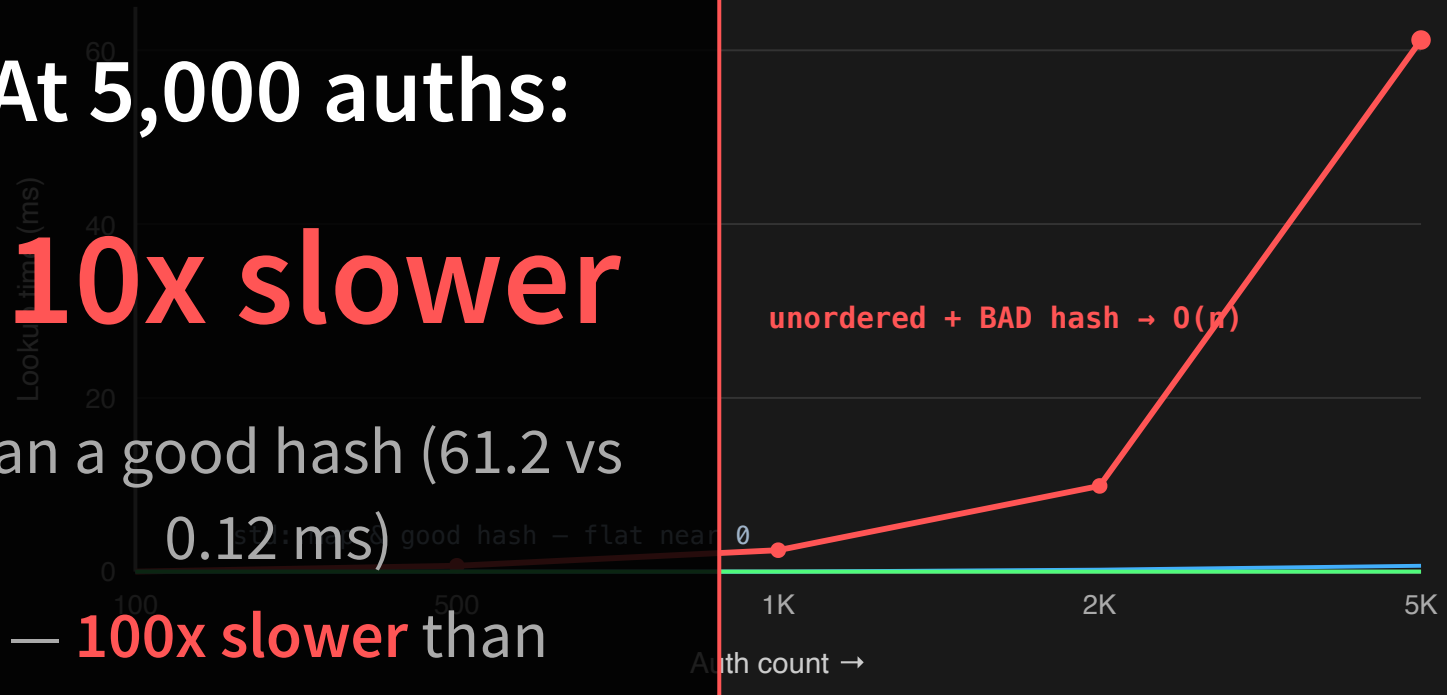
Collision Catastrophe: Results

At 5,000 auths:

510x slower

than a good hash (61.2 vs 0.12 ms)

— **100x slower** than `std::map`



src/06_pitfall2_collision_catastrophe_txn.cpp — representative run, regenerate on target (scripts/gen_numbers.sh)

With a bad hash:

`unordered_map` degrades to `O(n)`

which is **worse** than `map`'s `O(log n)`

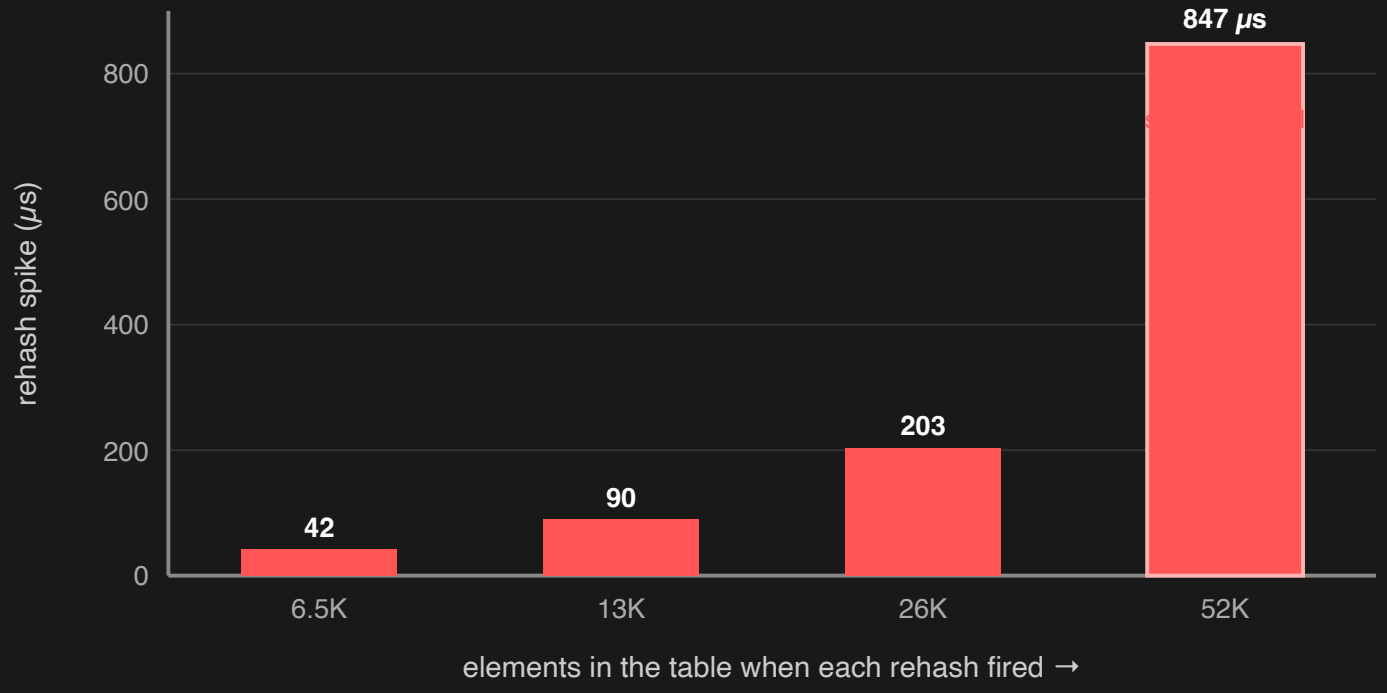
Your "fast" map is now 100x slower than the "slow" one.

Pitfall 3: The Hidden Rehash

What happens when the table gets too full?

> **Stop-the-world:** Every element is re-hashed and re-inserted.

Rehash Latency Spikes



src/07_pitfall3_rehash_txn.cpp

The Fix: `reserve()`

	Without reserve()	With reserve()
Avg insert time	0.42 us	0.31 us
Max insert time	2847.00 us	8.21 us
Rehash events	21	0

```
std::unordered_map<std::string, uint32_t> auth_cache;
auth_cache.reserve(expected_auth_count);   // one line, zero rehashes
```

src/08_pitfall3_reserve_fix_txn.cpp

When `O(no-no-no)` becomes a CVE

HashDoS: collisions as a weapon

The attack

1. Your server puts user input into an `unordered_map` (form fields, JSON keys, headers).
2. The attacker knows your hash — it's open source.
3. They send keys **crafted to all collide** into one bucket.
4. Every insert/lookup is now `O(n)`. One small request burns 100% CPU.

> A few hundred KB of crafted keys can pin a core for minutes.

The collision flood [LIVE]

```
// Weak hash an attacker can defeat by hand:
struct WeakSumHash {
    size_t operator()(const std::string& s) const {
        size_t h = 0;                          // anagrams collide!
        for (char c : s) h += (unsigned char)c;
        return h;
    }
};
```

Crafted keys	Strong hash	Weak hash (attacked)	Worst bucket
1,000	1.7 ms	4.7 ms	1,000
4,000	6.9 ms	55.8 ms	4,000
8,000	14.1 ms	529.7 ms	8,000

src/17_hashdos_attack.cpp — representative run, regenerate on target

This is not hypothetical

28C3, Dec 2011 — Klink & Wälde, "Efficient DoS on Web Application Platforms." One talk, many languages.

oCERT-2011-003 — PHP, Python, Ruby, Java, Node, ASP.NET all shipped emergency patches.

CVE-2011-3414 (ASP.NET) · CVE-2011-4815 (Ruby) · CVE-2012-1150 (Python) · Node.js cookie/querystring fixes

The fix the ecosystem chose: randomized hashing. Python adopted SipHash + a random seed (PEP 456).

Mitigations

```
// Per-process random salt: attacker can't precompute collisions
static const uint64_t kSalt = seed_from_os_rng();
struct SaltedHash {
    size_t operator()(const std::string& s) const {
        size_t h = std::hash<std::string>{}(s);
        return h ^ (kSalt + 0x9e3779b97f4a7c15ULL + (h<<6) + (h>>2));
    }
};
```

Randomized/keyed hash (SipHash) — the real fix. Salt changes every run.

The paradox: `reserve()` also helps — fewer rehashes = a smaller attack window.

Untrusted keys? Cap them, or fall back to a sorted container under load.

src/20_randomized_hashing.cpp

Beyond `unordered_map`

C++23 & Modern Alternatives

`std::flat_map` (C++23)

```cpp
#include <flat_map>

using BinTable_t = std::flat_map<
    std::string, BinInfo>;
BinTable_t bin_table;
bin_table["411111"] = {"CHASE", "VISA"};
auto it = bin_table.find("411111");
```

- Backed by two sorted `std::vector`s (contiguous memory)
- Cache-friendly iteration — blazing fast for read-heavy workloads
- Trade-off: O(n) insert, but BIN tables load *once* at startup
- Think: `std::map` meets `std::vector`

src/09_cpp23_flat_map_txn.cpp

Heterogeneous Lookup (C++20)

```cpp
struct StringHash {
    using is_transparent = void;  // Magic marker!

    size_t operator()(std::string_view sv) const {
        return std::hash<std::string_view>{}(sv);
    }
    size_t operator()(const std::string& s) const {
        return std::hash<std::string>{}(s);
    }
};

std::unordered_map<std::string, MerchantInfo,
                   StringHash, StringEqual> cache;

// Traditional: allocates temporary std::string every time
auto it = cache.find(std::string(sv_from_iso_message));

// Transparent: zero allocation!
auto it = cache.find(sv_from_iso_message);  // Direct!
```

Honest caveat: for a 3-char MML tag, **SSO** means the temporary never hit the heap — no win there. The payoff is on *longer* keys (merchant IDs, JSON field paths) that exceed SSO.

src/10_cpp20_heterogeneous_lookup_txn.cpp

Iterator Stability: When to Use `std::map`

```cpp
using AuthCache_t = std::unordered_map<
    std::string, AuthRecord>;
AuthCache_t auth_cache;
auth_cache["TXN_0001"] = {"AUTH_A1B2", 5000};

AuthRecord* ptr = &auth_cache["TXN_0001"];

// New auths flood in, triggering rehash...
for (int i = 3; i < 20; ++i)
    auth_cache["TXN_" + std::to_string(i)] = { /* ... */ };

// pending_reversal is NOW DANGLING!
```

`unordered_map`: rehash invalidates **ALL** pointers/iterators

`std::map`: insertion **NEVER** invalidates existing references

Open Addressing

Why your hot path might not want `std::unordered_map`

Why chaining hurts

Standard `unordered_map` = bucket array → **heap-allocated nodes** in a linked list.

Every collision step is a **pointer chase** to a random address → likely **`cache miss`**.

Remember: the standard's bucket + stability guarantees *force* this layout.

```
bucket[] → node@0x7f3a → node@0x91c2 → node@0x4b08 →
                          ...
  (each arrow = a jump to somewhere else in RAM)
```

Open addressing: keys live *in* the array

No nodes. On collision, **probe the next slot** (linear / quadratic / SwissTable groups).

+ Contiguous memory, cache-friendly, no per-element allocation.

– Tombstones on erase, sensitive to load factor, and…

– references invalidate on *any* insert (a flat map rehashes constantly).

The real-world options

`boost::unordered_flat_map` — drop-in-ish, Boost 1.81+

`absl::flat_hash_map` — Google SwissTable

`ankerl::unordered_dense` — header-only, very fast

1M BIN lookups	`unordered_map`	flat hash map
random find	9.8 ms	~2–5 ms

src/13_open_addressing_compare.cpp — regenerate with Boost on target (~2x+; fallback shown here)

The catch

Flat maps invalidate pointers/references on **every insert**.

Faster — but you give up the one thing `std::map` guaranteed.

It's the iterator-stability pitfall again, dialed to 11.

Memory

Where `O(1)` goes to die: the cache

Two layouts

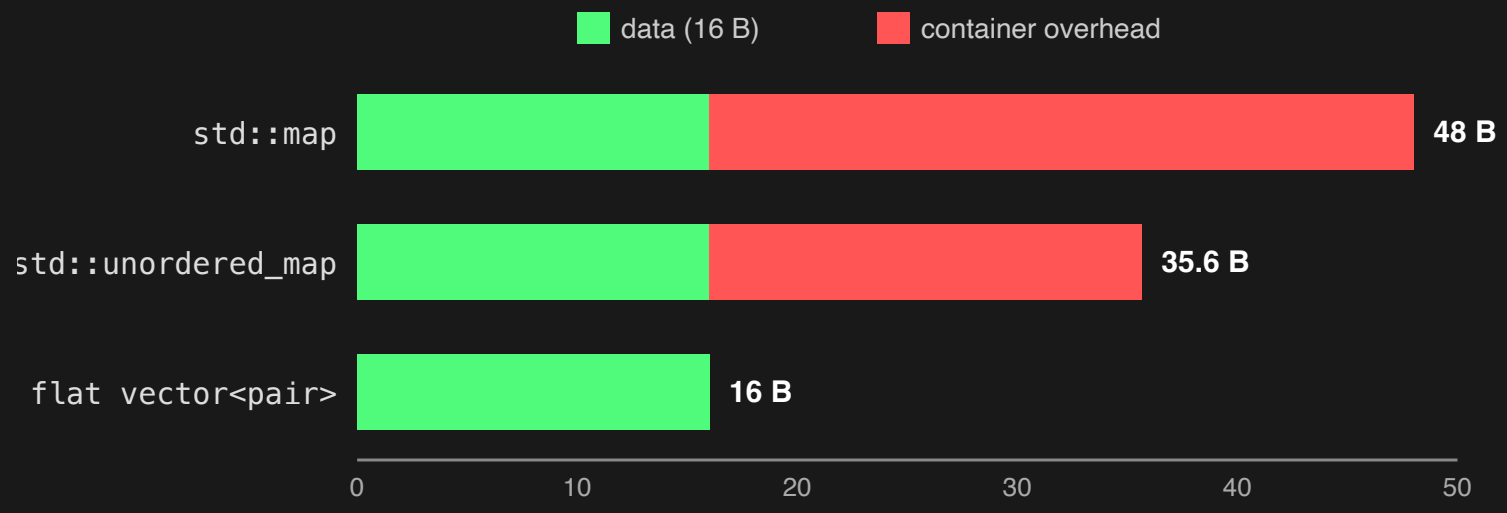
Node-based (`map`, `unordered_map`)

```
[bucket]→node→node
scattered across the heap
+ allocation header per
node
```

Contiguous (`flat_map`, flat hash)

```
[k|k|k|k|k|k|k|k]
one array, one allocation
prefetcher-friendly
```

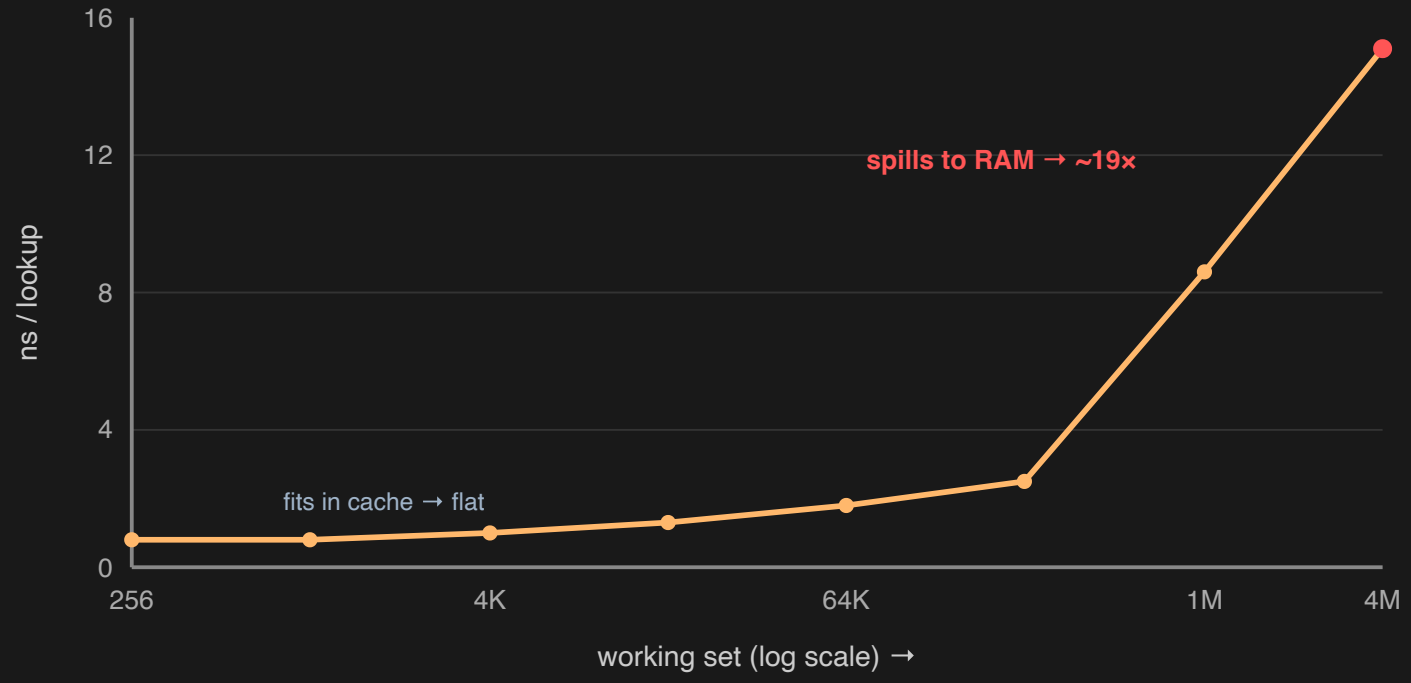
Per-element overhead (measured, not quoted)



16 data bytes/elem. A counting allocator tallies real heap bytes — run it on your stdlib.

src/14_memory_overhead.cpp

The cache cliff



Same `O(1)` — but the constant grew **~19x** as the table left cache.

src/15_cache_effects.cpp

Two more levers

Allocators: `std::pmr::unordered_map` + a pool/monotonic buffer keeps nodes close together — clawing back some locality. Great for fixed-size auth records.

Stdlib matters: libstdc++ vs libc++ vs MSVC differ on bucket growth, default `max_load_factor`, and node layout.

Same code, GCC vs Clang on this laptop → different memory + timings. Benchmarks don't transfer.

Profiling & Production

Diagnosing hash maps in the wild

Measure cache misses, don't guess

```
# Linux (in Docker on the Mac):
perf stat -e cache-misses ./19_perf_workload chained
perf stat -e cache-misses ./19_perf_workload flat
# macOS: Instruments "Counters", or  mperf -- ./19_perf_workload flat
```

Chained: high cache-miss rate (node chasing).

Flat: sequential access, far fewer misses.

(Captured ahead of time — never profile live on stage.)

src/19_perf_workload.cpp

Find the hot bucket at runtime

```
for (size_t b = 0; b < m.bucket_count(); ++b)
    if (m.bucket_size(b) > 8 * mean)
        log_hot_bucket(b, m.bucket_size(b));   // hash problem, not load
```

```
--- clustering hash ---
  occupied buckets=100   max bucket=500   (ideal ~0.6)
  hot buckets (> 8): 100   <-- investigate the hash!
```

src/16_bucket_inspection.cpp

The container is rarely your bottleneck

Real auth service: each lookup table is behind a shared lock, unlocked *per lookup*, on every worker thread. 2M lookups, 4 threads:

getShared() per lookup	`std::map`	`unordered_map`
contended	501 ms	548 ms

Lock-bound — the container choice is **noise**. Now hoist the lock out of the loop:

getShared() hoisted	`std::map`	`unordered_map`
lock once / thread	48 ms	20 ms

10–27x from moving the lock — *then* the container matters. Profile first.

src/21_locking_dominates.cpp — representative run, regenerate on target

Concurrency: it's not thread-safe

`unordered_map` — concurrent read+write is **UB**. Read-only concurrent access is fine.

One global mutex serializes everything. **Sharding** (N maps, N locks) restores parallelism:

4 threads, 75% reads	single mutex	sharded ×16
throughput	149.6 ms	69.5 ms (2.15x)

Production: `tbb::concurrent_hash_map`, `folly::ConcurrentHashMap`. And a rehash mid-flight is especially dangerous.

src/18_sharded_concurrent.cpp

Case Study

Auditing a real auth service

The question

A real payment-auth service. ~10 maps. **Every one is `std::map`.**

"Should we switch them all to `unordered_map`?"

Let's run them through the framework instead of swapping on reflex.

One map at a time

Map	Role	Verdict	Why
field validations	read-mostly lookup	switch	per-request `find()`, string key
xml↔mml mappings	read-mostly lookup	switch	point lookups only
request tracking	grows/shrinks w/ load	keep	would **rehash-spike** under load (Pitfall 3)
transient request map	iterated → MML+SHA1	never	order is the wire format (Pitfall 0)
active merchants	`set`, custom key	keep	needs a hand-written hash; 1 lookup/req

Every pitfall in this talk showed up in one small service.

The swap that mattered wasn't a swap.

The real win was **hoisting the lock out of the loop** — 10–27x.

The container change that started it all? In the noise.

The Decision Framework

Choosing the Right Container

Three Rules

`std::vector` is your default sequence.

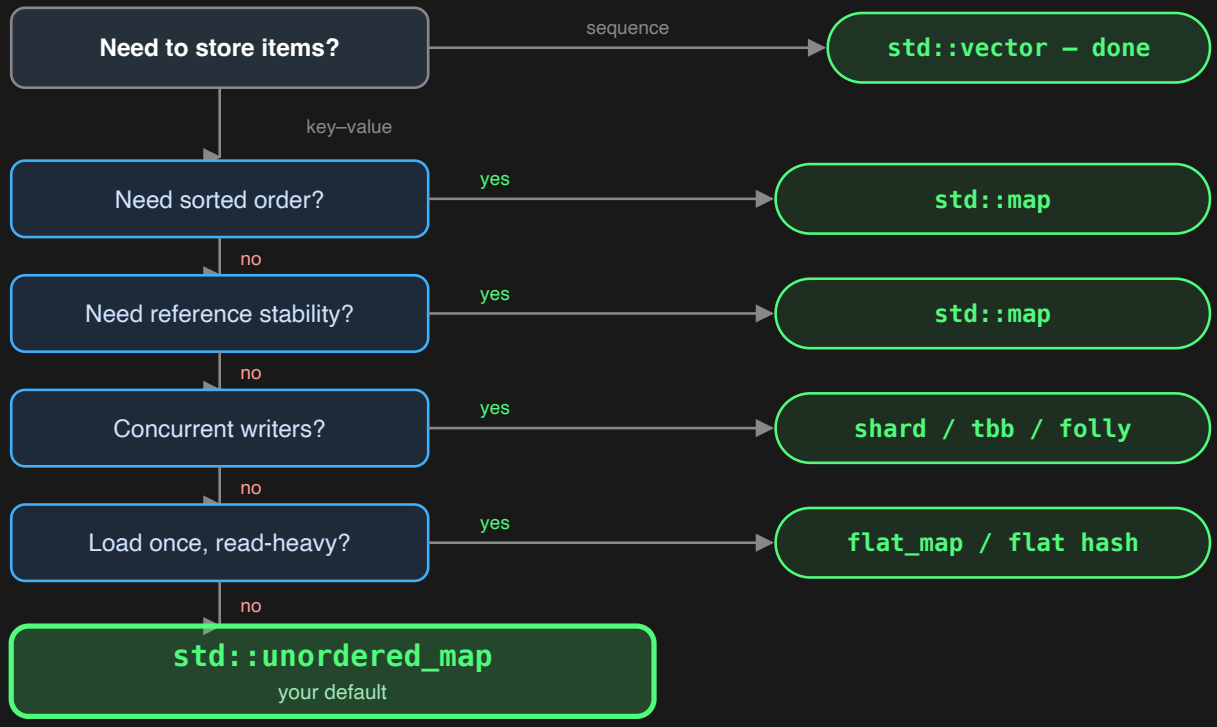
`std::unordered_map` is your default associative container.

It is `O(1)` fast but `O(n)` dangerous.

0. **Order** → don't swap a map you iterate for output
1. **Bad Hashes** → use `hash_combine`
2. **Collisions** → hash *ALL* fields
3. **Rehashing** → call `reserve()`

…and **measure** — the container is rarely your bottleneck.

Your New Decision Tree



Pro tips when you land on `unordered_map`: Custom key? → write a good hash (`hash_combine`). Know the size? → call `reserve()`. Read-heavy + load-once? → consider `flat_map` or a flat hash map.

src/12_decision_framework_txn.cpp

Stop using `std::map` "just in case."

Start with `std::unordered_map` — then master its pitfalls: hash all fields, call `reserve()`, know when a flat map wins.

Never swap a map whose *order* you depend on. And **measure** — the container is rarely the bottleneck.

Write faster — and *correct* — modern C++.

Thank You